

In [ ]:

## Разделы:

- [Современные методы обучения нейронной сети и обратное распространение ошибки](#)
- [Обучение модели нейронной сети, алгоритм обратного распространения ошибки](#)
  - [Проблема обучения модели нейронной сети](#)
  - [Проблема поиска градиента](#)
- [Дифференцируемое программирование и реализация обратного распространения ошибки](#)
  - [Автоматическое дифференцирование в PyTorch](#)

-

- [к оглавлению](#)

```
In [26]: # загружаем стиль для оформления презентации
from IPython.display import HTML
from urllib.request import urlopen
html = urlopen("file:./lec_v2.css")
HTML(html.read().decode('utf-8'))
```

Out[26]:

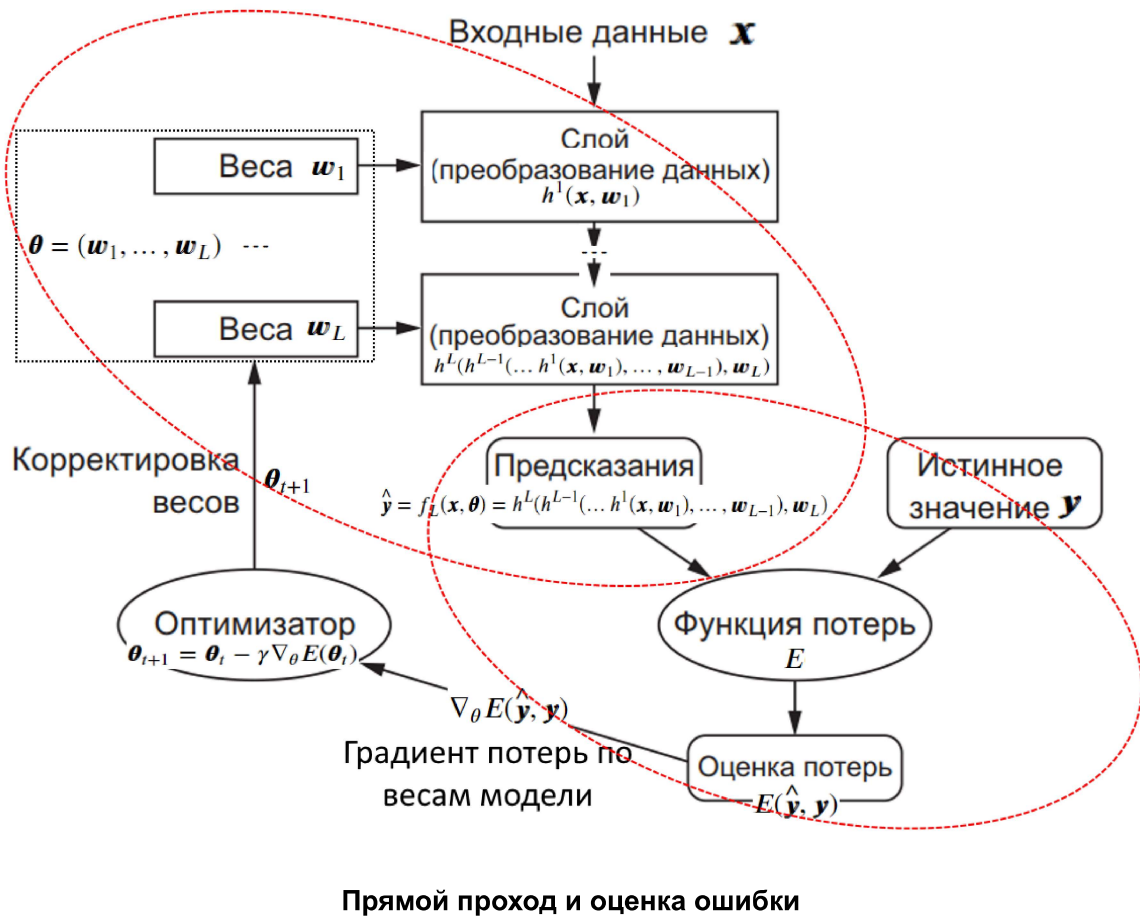
## Современные методы обучения нейронной сети и обратное распространение ошибки

- [к оглавлению](#)

## Применение тензоров: прямое распространение сигналов и оценка ошибки

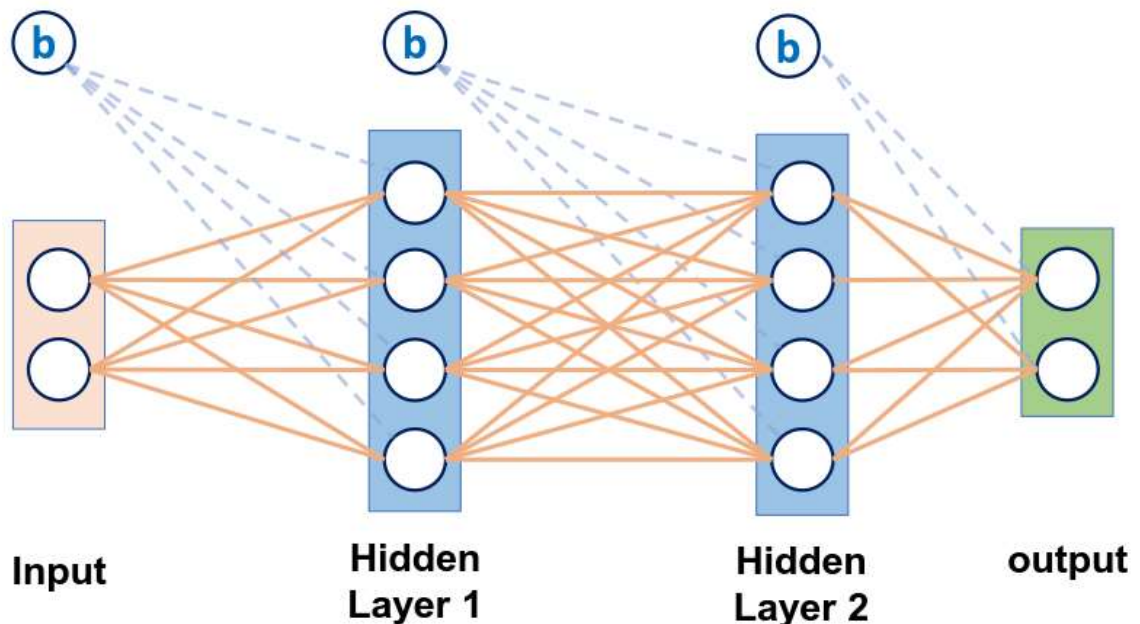
### Постановка задачи

- У нас есть набор данных  $D$ , состоящий из пар  $(\mathbf{x}, \mathbf{y})$ , где  $\mathbf{x}$  - признаки, а  $\mathbf{y}$  - правильный ответ.
- Модель сети  $f_L$ , имеющей  $L$  слоев с весами  $\boldsymbol{\theta}$  (совокупность весов нейронов из всех слоев) на этих данных делает некоторые предсказания  $\hat{\mathbf{y}} = f_L(\mathbf{x}, \boldsymbol{\theta})$
- Задана функция ошибки  $E$ , которую можно подсчитать на каждом примере:  $E(f_L(\mathbf{x}, \boldsymbol{\theta}), \mathbf{y})$  (например, это может быть квадрат или модуль отклонения  $\hat{\mathbf{y}}$  от  $\mathbf{y}$  в случае регрессии или перекрестная энтропия в случае классификации)
- Тогда суммарная ошибка на наборе данных  $D$  будет функцией от параметров модели:  $E(\boldsymbol{\theta})$  и определяется как  $E(\boldsymbol{\theta}) = \sum_{(\mathbf{x}, \mathbf{y}) \in D} E(f_L(\mathbf{x}, \boldsymbol{\theta}), \mathbf{y})$



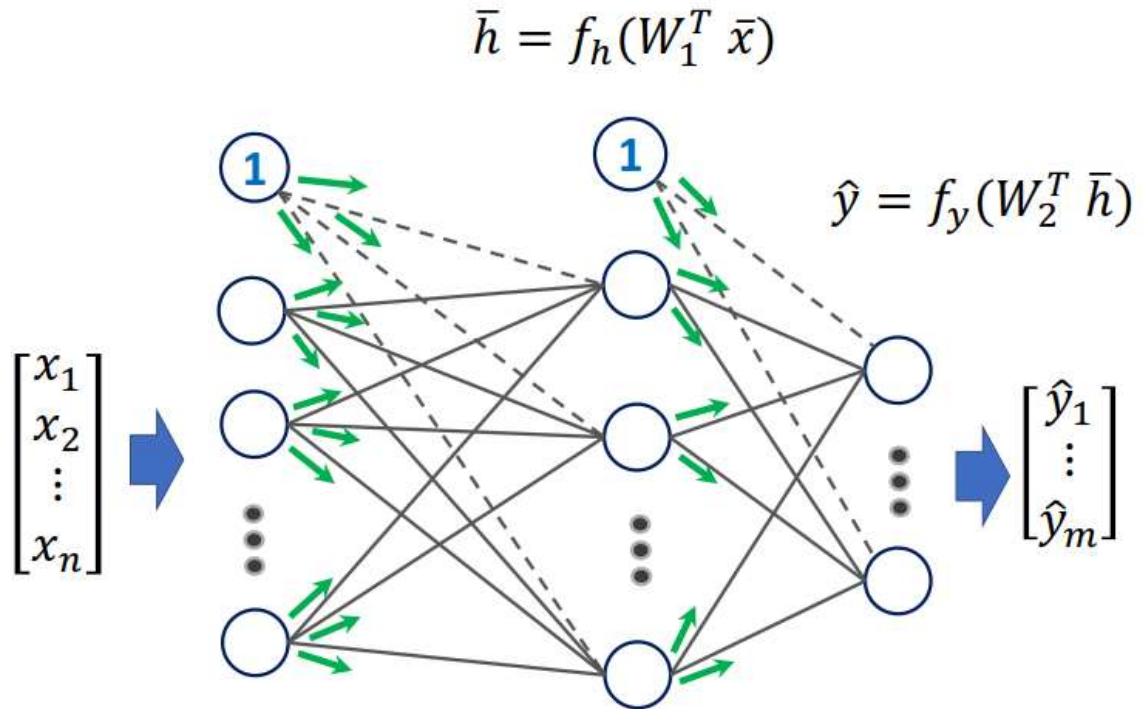
### Прямое распространение сигналов

- Модель нейронной сети это иерархия (она может быть простой и очень сложной) связанных (последовательно применяемых) функций слоев:
  - т.е. модель сети  $f_L$  может быть представлена как суперпозиция из  $L$  слоев  $h^i, i \in \{1, \dots, L\}$ , каждый из которых параметризуется своими весами  $\mathbf{w}_i$ :
 
$$f_L(\mathbf{x}, \boldsymbol{\theta}) = f_L(\mathbf{x}, \mathbf{w}_1, \dots, \mathbf{w}_L) = h^L(h^{L-1}(\dots h^1(\mathbf{x}, \mathbf{w}_1), \dots, \mathbf{w}_{L-1}), \mathbf{w}_L)$$



### пример модели сети: многослойный перцептрон с двумя скрытыми слоями

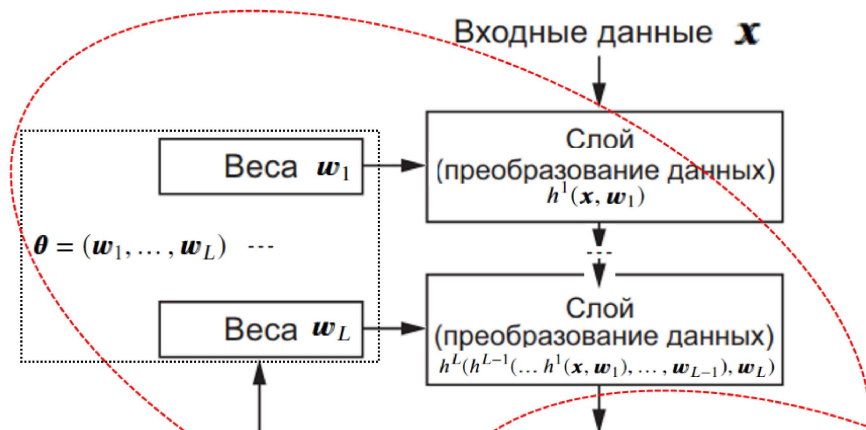
- Прямое распространение сигналов по модели (в частности: нейронной сети) реализуется с помощью **прямого прохода (forward pass)**: входящая информация (вектор  $\mathbf{x}$ ) распространяется через сеть  $f_L$  с учетом весов связей  $\boldsymbol{\theta}$ , рассчитывается выходной вектор  $\hat{\mathbf{y}} = f_L(\mathbf{x}, \boldsymbol{\theta})$ .
  - Каждый слой нейронной сети - это последовательно применяемая функция слоя, которая рассчитывается при помощи операций с тензорами.



14

### пример прямого прохода

- Последовательность операций с тензорами используется для расчета результата:
  - Прямой проход (forward pass): входящая информация (вектор  $\mathbf{x}$ ) распространяется через сеть с учетом весов связей, рассчитывается выходной вектор  $\hat{\mathbf{y}} = f_L(\mathbf{x}, \boldsymbol{\theta}) = h^L(h^{L-1}(\dots h^1(\mathbf{x}, \mathbf{w}_1), \dots, \mathbf{w}_{L-1}), \mathbf{w}_L)$
  - Оценки ошибки  $E(\hat{\mathbf{y}}, \mathbf{y})$  на множестве правильных ответов:  $\mathbf{y}$ .



Почему для реализации модели ИНС используют тензоры?

Рассмотрим примеры моделей (веса):

- 1 персептрон (4 входа + константа(смещение, bias) ): python  
`inputs = torch.tensor([1.0, 2.0, 3.0, 2.5])`  
`weights = torch.tensor([0.2, 0.8, -0.5, 1.0, 2.0])`
- 1 слой персептронов из 3 нейронов (ось 0), каждый с 4 входами и константой (ось 1):

```
inputs = torch.tensor([1.0, 2.0, 3.0, 2.5])
weights = torch.tensor([[0.2, 0.8, -0.5, 1.0, 2.0],
                        [0.5, -0.91, 0.26, -0.5, 3.0],
                        [-0.26, -0.27, 0.17, 0.87, 0.5]])
```

- 2 слоя персептронов из 3 нейронов и из 2 нейронов

```
inputs = torch.tensor([1.0, 2.0, 3.0, 2.5])
weights_layer1 = torch.tensor([[0.2, 0.8, -0.5, 1.0, 2.0],
                               [0.5, -0.91, 0.26, -0.5, 3.0],
                               [-0.26, -0.27, 0.17, 0.87, 0.5]])
# выход слоя 1 - вход слоя 2
weights2_layer2 = torch.tensor([[0.8, 0.5, 1.1, 2.0],
                                [0.4, 0.26, -0.4, 3.0],
                                [-0.2, 0.27, 0.17, 0.5]])
```

- набор входных векторов (batch):

```
inputs = torch.tensor([[1.0, 2.0, 3.0, 2.5],
                       [-1.1, 3.0, 2.1, 0.8],
                       [-2.0, 1.3, 0.1, -1.8],
                       [-1.3, 3.2, 1.1, 0.6]])
weights = torch.tensor([[0.2, 0.8, -0.5, 1.0, 2.0],
                        [0.5, -0.91, 0.26, -0.5, 3.0],
                        [-0.26, -0.27, 0.17, 0.87, 0.5]])
```

```
In [60]: import torch
```

```
In [70]: # Модель линейной регрессии (с несколькими параметрами)
#  $f = X * w$ 

# Данные для обучения:
# признаки X: рассматривается 4 наблюдения (ось 0) и 2 признака (ось 1):

# вариант исходных данны #1 (2й признак всегда равен 0):
# X = torch.tensor([[1., 0.],
#                   [2., 0.],
#                   [3., 0.],
#                   [4., 0.]], dtype=torch.float32) # Size([4, 2])

# вариант исходных данны #2 (2й признак используется и существенно больше 1го)
X = torch.tensor([[1., 40.],
                  [2., 30.],
                  [3., 20.],
                  [4., 10.]], dtype=torch.float32) # Size([4, 2])

print(f'Матрица X: \n{X}')
print(f'X.size = {X.size()}')

# истинное значение весов (используется только для получения обучающих правил)

# вариант #1:
# w_ans = torch.tensor([2., 0.], dtype=torch.float32)

# вариант #2:
w_ans = torch.tensor([2., 1.], dtype=torch.float32)

# Y - правильные ответы:
Y = X @ w_ans

torch.set_printoptions(precision=5) # точность вывода на печать значений тензоров
print(f'w true value = {w_ans}, Y = {Y}')
```

Матрица X:

```
tensor([[ 1., 40.],
        [ 2., 30.],
        [ 3., 20.],
        [ 4., 10.]])
```

```
X.size = torch.Size([4, 2])
```

```
w true value = tensor([2., 1.]), Y = tensor([42., 34., 26., 18.]')
```

```
In [71]: # model (модель, в нашем случае: линейная регрессия)

# изначальное значение весов w
w = torch.tensor([0.0, 0.0], dtype=torch.float32, requires_grad=False)

print(f'w:\n{w}')

# прямое распространение (здесь фактически описывается модель):
def forward(X):
    return X @ w # Size([4])

w:
tensor([0., 0.]
```

## Обучение модели нейронной сети, алгоритм обратного распространения ошибки

- [к оглавлению](#)

### Проблема обучения модели нейронной сети

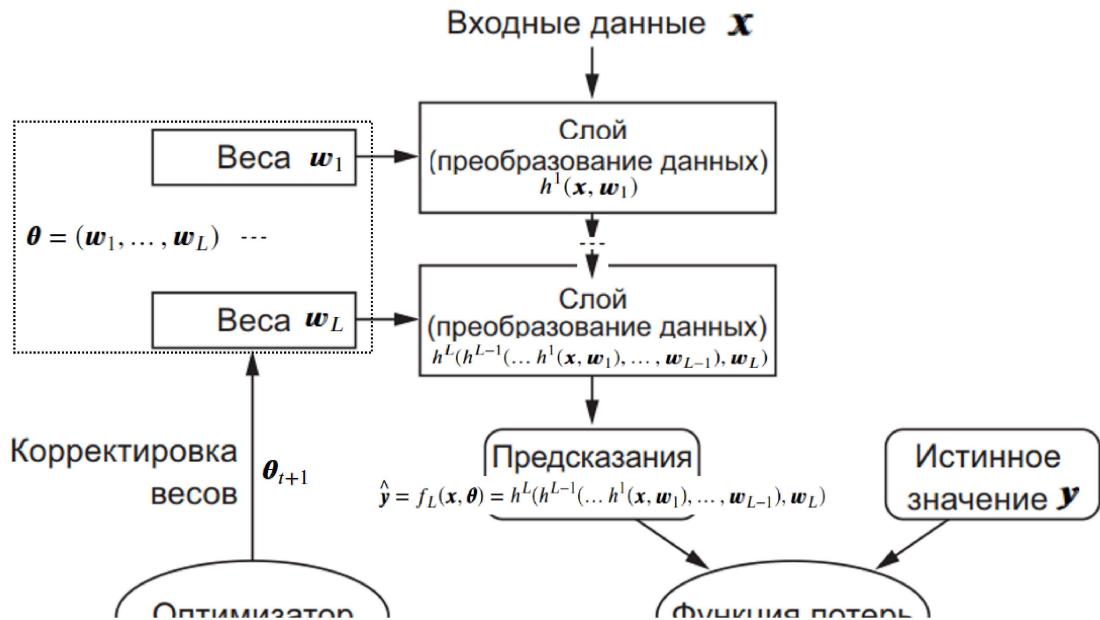
- [к оглавлению](#)

#### Проблема обучения модели нейронной сети: общий взгляд

- **основная проблема** это не применение модели к входным данным  $\mathbf{x}$  и оценка ошибки на правильных ответах  $\mathbf{y}$ , а **обучение модели** (определение наилучших параметров модели  $\boldsymbol{\theta}$ ).
  - В случае нейронной сети обучение сводится к поиску весов слоев сети  $\boldsymbol{\theta} = (\mathbf{w}_1, \dots, \mathbf{w}_L)$ , которые в совокупности являются параметрами модели  $\boldsymbol{\theta}$ .
- Формально: цель обучения - найти оптимальное значение параметров  $\boldsymbol{\theta}^*$ , минимизирующих ошибку на обучающей выборке  $D$ :

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} E(\boldsymbol{\theta}) = \arg \min_{\boldsymbol{\theta}} \sum_{(\mathbf{x}, \mathbf{y}) \in D} E(f_L(\mathbf{x}, \boldsymbol{\theta}), \mathbf{y})$$

- Т.е. задача обучения сводится к задаче оптимизации.
  - На самом деле **все сложнее**: хороший результат на  $D$  может плохо обобщаться (модель может давать низкое качество на другой выборке из той же генеральной совокупности) - **проблема переобучения**.

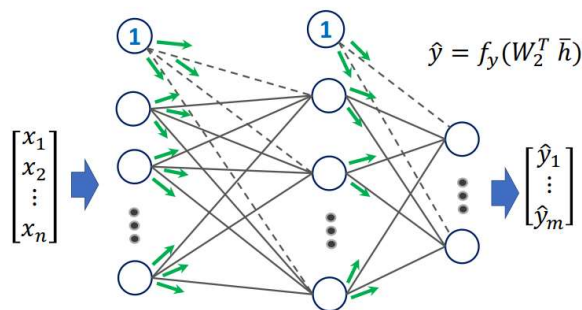


### Прямой проход и оценка ошибки

- **Прямой проход** (forward pass): входящая информация (вектор  $\mathbf{x}$ ) распространяется через сеть с учетом весов связей, рассчитывается выходной вектор

$$\hat{\mathbf{y}} = f_L(\mathbf{x}, \boldsymbol{\theta}) = h^L(h^{L-1}(\dots h^1(\mathbf{x}, \mathbf{w}_1), \dots, \mathbf{w}_{L-1}), \mathbf{w}_L)$$

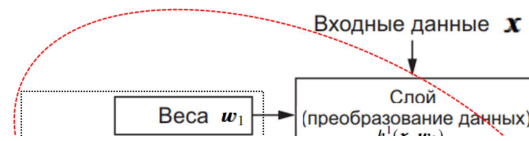
$$\bar{h} = f_h(W_1^T \bar{x})$$



### пример прямого прохода

- **Оценки ошибки**  $E(\hat{\mathbf{y}}, \mathbf{y})$  на множестве правильных ответов:  $\mathbf{y}$ .

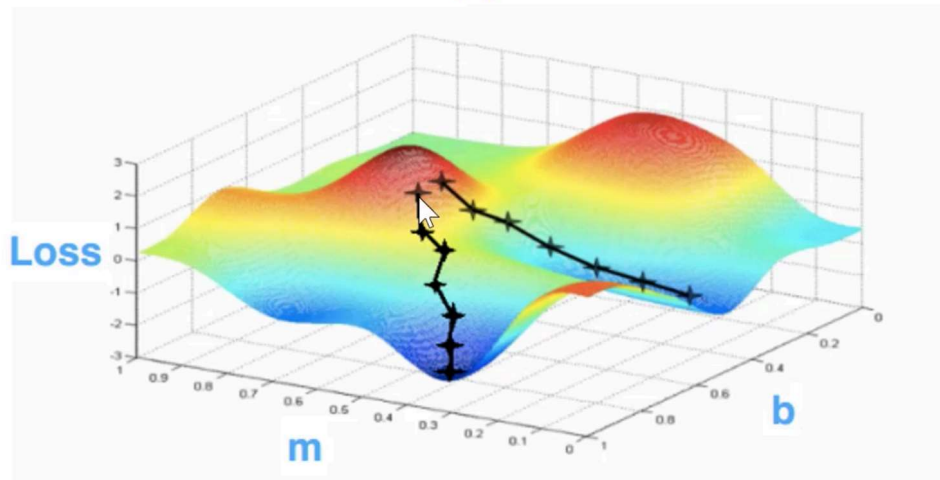




### Задача оптимизации

- Задача: корректировка весов сети (параметров модели  $\theta$ ) на основе информации об ошибке на обучающих примерах  $E(\hat{y}, y)$ .
  - Решение: использовать методы оптимизации, основанные на **методе градиентного спуска**.
- Метод градиентного спуска** - метод нахождения локального экстремума (минимума или максимума) функции с помощью движения вдоль градиента. В нашем случае шаг метода градиентного спуска выглядит следующим образом:
 
$$\theta_t = \theta_{t-1} - \gamma \nabla_{\theta} E(\theta_{t-1}) = \theta_{t-1} - \gamma \sum_{(x,y) \in D} \nabla_{\theta} E(f_L(x, \theta), y)$$
- Выполнение на каждом шаге градиентного спуска суммирование по всем  $(x, y) \in D$  **обычно слишком неэффективно**
- Для выпуклых функций **задача локальной оптимизации** - найти локальный минимум (максимум) автоматически превращается в **задачу глобальной оптимизации** - найти точку, в которой достигается наименьшее (наибольшее) значение функции, то есть самую низкую (высокую) точку среди всех.
- Оптимизировать веса одного перцептрона - выпуклая задача, но **для большой нейронной сети целевая функция не является выпуклой**.

$f(x) = \text{nonlinear function of } x$



### Пример работы градиентного спуска для функции двух переменных

- У нейронных сетей функция ошибки может задавать **очень сложный ландшафт** с огромным числом локальных максимумов и минимумов. Это свойство необходимо для обеспечения **выразительности нейронных сетей**, позволяющей им решать так много разных задач.
- для использования методов, основанных на методе градиентного спуска **необходимо знать градиент функции потерь по параметрам модели**:



$\nabla_{\theta} E(f_L(\mathbf{x}, \boldsymbol{\theta}), \mathbf{y})$ . Этот градиент определяет вектор ("направление") изменения параметров.

```
In [72]: # model (модель, в нашем случае: линейная регрессия)

# изначальное значение весов w
w = torch.tensor([0.0, 0.0], dtype=torch.float32, requires_grad=False)

print(f'w:\n{w}')

# прямое распространение:
def forward(X):
    return X @ w # Size([4])

# Loss = MSE (функция потерь, в нашем случае: средняя квадратичная ошибка)
def loss(y, y_pred):
    return ((y_pred - y)**2).mean() # Size([])

# градиент:
# рассчитан аналитически по модели и функции потерь:
# J = MSE = 1/N * (w*x - y)**2
# dJ/dw = 1/N * 2 * (w*x - y) * x
def gradient(x, y, y_pred):
    #     print(f''y = {y},
    #     y_pred = {y_pred},
    #     (2* (y_pred - y)).unsqueeze(1) = {(2* (y_pred - y)).unsqueeze(1)},
    #     x = {x},
    #     ((2* (y_pred - y)).unsqueeze(1) * x) = {(2* (y_pred - y)).unsqueeze(1)
    #     ((2* (y_pred - y)).unsqueeze(1) * x).mean(dim=0) = {(2* (y_pred - y)).u
    return ((2* (y_pred - y)).unsqueeze(1) * x).mean(dim=0)

w:
tensor([0., 0.]
```

```
In [73]: learning_rate = 0.0013

# predict = forward pass
y_pred = forward(X)

# Loss
l = loss(Y, y_pred)

# calculate gradients
dw = gradient(X, Y, y_pred)

# update weights
w -= learning_rate * dw
```

```

In [74]: # Training

# вариант #1:
# learning_rate = 0.05
# n_iters = 20 + 1

# вариант #2:
learning_rate = 0.0013
n_iters = 1000 + 1

# основной цикл:
for epoch in range(n_iters):
    # predict = forward pass
    y_pred = forward(X)

    # loss
    l = loss(Y, y_pred)

    # calculate gradients
    dw = gradient(X, Y, y_pred)

    # update weights
    w -= learning_rate * dw

    if epoch % 5 == 0:
        print(f'epoch {epoch}: w = {w}, y_pred = {y_pred}, loss = {l:.8f}\ngra

```

```

epoch 0: w = tensor([0.04740, 0.08853]), y_pred = tensor([88.56900, 66.638
00, 44.70700, 22.77600]), loss = 901.66833496
gradient = tensor([ 93.53500, 1631.90002])
epoch 5: w = tensor([0.27269, 1.96004]), y_pred = tensor([9.90674, 7.6027
2, 5.29869, 2.99467]), loss = 595.12457275
gradient = tensor([ -103.50652, -1319.86426])
epoch 10: w = tensor([0.26484, 0.43254]), y_pred = tensor([73.32084, 55.41
520, 37.50956, 19.60392]), loss = 393.66198730
gradient = tensor([ 57.54780, 1070.75989])
epoch 15: w = tensor([0.43660, 1.65862]), y_pred = tensor([21.67870, 16.68
858, 11.69846, 6.70834]), loss = 261.16906738
gradient = tensor([ -71.50771, -865.57104])
epoch 20: w = tensor([0.45417, 0.65508]), y_pred = tensor([63.23888, 48.05
265, 32.86644, 17.68021]), loss = 173.95428467
gradient = tensor([ 34.33218, 702.63275])
epoch 25: w = tensor([0.58835, 1.45793]), y_pred = tensor([29.32578, 22.64
829, 15.97079, 9.29329]), loss = 116.47224426
gradient = tensor([ -50.14605, -567.58569])
epoch 30: w = tensor([0.62025, 0.79825]), y_pred = tensor([56.55405, 43.22
346, 30.00000, 16.55000]), loss = 70.53340600

```

## Проблема поиска градиента

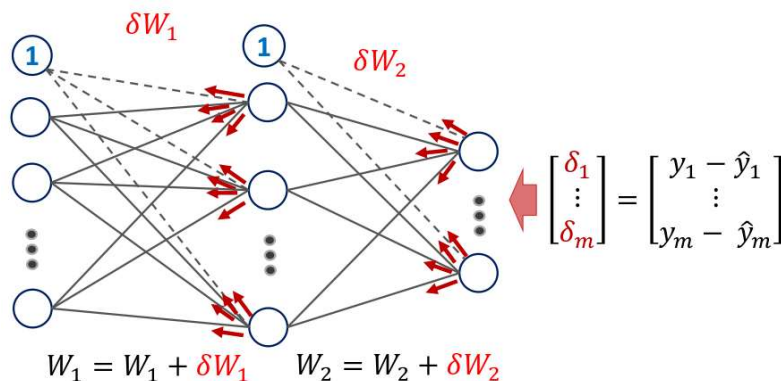
- [к оглавлению](#)

- Проблема: как найти градиент для нейронной сети:  $\nabla_{\theta} E(f_L(\mathbf{x}, \theta), \mathbf{y})$ ?



### Проблема поиска градиента в общей логике обучения нейронной сети

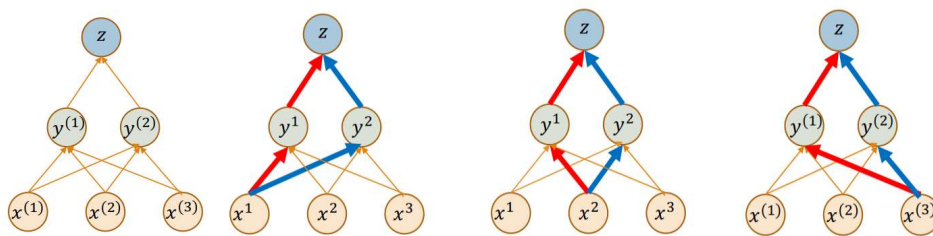
- Для решения этой задачи и используется **алгоритм обратного распространения ошибки** (backpropagation). Суть алгоритма:
  - рассчитывается ошибка между выходным вектором сети  $\hat{\mathbf{y}}$  и правильным ответом обучающего примера  $\mathbf{y}$
  - ошибка распространяется от результата к источнику (в обратную сторону) для корректировки весов



### Рассчет градиента суперпозиции двух функций нескольких переменных

- Сначала рассмотрим подзадачу: как рассчитать градиент для  $f_L(\mathbf{x}, \mathbf{w}_1, \dots, \mathbf{w}_L) = h^L(h^{L-1}(\dots, h^1(\mathbf{x}, \mathbf{w}_1), \dots, \mathbf{w}_{L-1}), \mathbf{w}_L)$ ?
- Для этого нам нужно будет **рассчитывать градиент суперпозиции (сложной функции)** состоящей из последовательного применения функций слоев  $h^i$ .
- Вспомним, как рассчитать производную (градиент) суперпозиции нескольких функций.
  - Пусть  $z = f(y)$ ,  $y = g(x)$
  - Тогда производная суперпозиции функций (правило дифференцирования сложной функции (chain rule)):  $\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$
  - Если  $\mathbf{x} \in \mathbb{R}^n$ ,  $\mathbf{y} \in \mathbb{R}^m$ , а  $\mathbf{z} \in \mathbb{R}$ , то:  $\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}$

## Примеры расчета градиента суперпозиции двух функций нескольких переменных:



$$\frac{dz}{dx^1} = \frac{dz}{dy^1} \frac{dy^1}{dx^1} + \frac{dz}{dy^2} \frac{dy^2}{dx^1} \quad \frac{dz}{dx^2} = \frac{dz}{dy^1} \frac{dy^1}{dx^2} + \frac{dz}{dy^2} \frac{dy^2}{dx^2} \quad \frac{dz}{dx^3} = \frac{dz}{dy^1} \frac{dy^1}{dx^3} + \frac{dz}{dy^2} \frac{dy^2}{dx^3}$$

Т.е. нам нужны градиенты по всем возможным путям (рассмотренным в обратном порядке) зависимостей переменных.

Запись этой же задачи в векторной нотации:

$$\bullet \quad \frac{dz}{dx} = \nabla_x(z) = \begin{pmatrix} \frac{\partial z}{\partial x_1} \\ \dots \\ \frac{\partial z}{\partial x_n} \end{pmatrix} = \left( \frac{dy}{dx} \right)^T \cdot \nabla_y(z) = J(y(x))^T \cdot \nabla_y(z) = J(y(x))^T \cdot \begin{pmatrix} \frac{\partial z}{\partial y_1} \\ \dots \\ \frac{\partial z}{\partial y_m} \end{pmatrix}$$

• Где  $J$  это Якобиан:

$$J(y(x)) = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \dots & \frac{\partial y_m}{\partial x_n} \end{pmatrix}$$

### Задача поиска градиента: $\nabla_{\theta} E(f_L(\mathbf{x}, \theta), \mathbf{y})$

- Перейдем от  $f_L$  к последовательному расчету функций слоев  $h^i$ :  
 $\nabla_{\theta} E(f_L(\mathbf{x}, \theta), \mathbf{y}) = \nabla_{\mathbf{w}_i} E(f_L(\mathbf{x}, \mathbf{w}_1, \dots, \mathbf{w}_L), \mathbf{y}) = \nabla_{\mathbf{w}_i} E(h^L(h^{L-1}(\dots h^1(\mathbf{x}, \mathbf{w}_1), \dots, \mathbf{w}_L))$
- Обозначим через  $\mathbf{a}^l$  результат расчета функции активации на слое  $l$ :  $\mathbf{a}^l = h^l(\mathbf{x}_l, \mathbf{w}_l)$ .  
 Тогда:  $\mathbf{x}_{l+1} = \mathbf{a}_l$  (вход следующего слоя является результатом расчета функции активации предыдущего слоя)
- Тогда можно записать:  $\nabla_{\theta} E(f_L(\mathbf{x}, \theta), \mathbf{y}) = \nabla_{\theta} E(\mathbf{a}^L, \mathbf{y})$ . Функция потерь  $E(\mathbf{a}^L, \mathbf{y})$  зависит от  $\mathbf{a}^L$ ,  $\mathbf{a}^L$  от  $\mathbf{a}^{L-1}$ , ...,  $\mathbf{a}^{l+1}$  от  $\mathbf{a}^l$
- Исходя из этого представления можно градиенты весов  $l$ -го слоя можно записать как:

$$\frac{\partial E}{\partial \mathbf{w}_l} = \frac{\partial E}{\partial \mathbf{a}_L} \cdot \frac{\partial \mathbf{a}_L}{\partial \mathbf{a}_{L-1}} \cdot \dots \cdot \frac{\partial \mathbf{a}_{l+1}}{\partial \mathbf{a}_l} \cdot \frac{\partial \mathbf{a}_l}{\partial \mathbf{w}_l}$$

- Произведение всех сомножителей кроме последнего является градиентом функции потерь по результатам расчета функции активации слоя  $l$ :

$$\frac{\partial E}{\partial \mathbf{a}_L} \cdot \frac{\partial \mathbf{a}_L}{\partial \mathbf{a}_{L-1}} \cdot \dots \cdot \frac{\partial \mathbf{a}_{l+1}}{\partial \mathbf{a}_l} = \frac{\partial E}{\partial \mathbf{a}_l}$$

- Тогда:

$$\frac{\partial E}{\partial \mathbf{w}_l} = \left( \frac{\partial \mathbf{a}_l}{\partial \mathbf{w}_l} \right)^T \cdot \frac{\partial E}{\partial \mathbf{a}_l}$$

для расчета  $\frac{\partial \mathbf{a}_l}{\partial \mathbf{w}_l}$  нам нужен только якобиан функции активации  $l$ -го слоя по параметрам слоя  $\mathbf{w}_l$ .

- Градиент функции потерь по результатам расчета функции активации слоя  $l$  может быть рассчитан рекурсивно по результатам слоя  $l$ , собственно тут и происходит

**обратное распространение:**  $\frac{\partial E}{\partial \mathbf{a}_l} = \left( \frac{\partial \mathbf{a}_{l+1}}{\partial \mathbf{a}_l} \right)^T \cdot \frac{\partial E}{\partial \mathbf{a}_{l+1}} = \left( \frac{\partial \mathbf{a}_{l+1}}{\partial \mathbf{x}_{l+1}} \right)^T \cdot \frac{\partial E}{\partial \mathbf{a}_{l+1}}$  для

расчета  $\frac{\partial \mathbf{a}_{l+1}}{\partial \mathbf{x}_{l+1}}$  нам нужен только якобиан функции активации  $l + 1$ -го слоя по входным значениям слоя  $\mathbf{x}_{l+1}$

- Т.е. чтобы проводить обратное распространение ошибки, нам на каждом слое (например  $l$ -м) нужно рассчитывать два якобиана:

- якобиан функции активации  $l$ -го слоя по параметрам слоя  $\frac{\partial \mathbf{a}_l}{\partial \mathbf{w}_l}$  - он позволит

рассчитать градиент  $\frac{\partial E}{\partial \mathbf{w}_l}$  и сделать очередной шаг градиентного спуска для

параметров этого слоя:  $\mathbf{w}_l^{t+1} = \mathbf{w}_l^t - \gamma \nabla_{\mathbf{w}_l} E(\mathbf{w}^t) = \mathbf{w}_l^t - \gamma \frac{\partial E(\mathbf{w}^t)}{\partial \mathbf{w}_l}$

- якобиан функции активации  $l$ -го слоя по входным значениям слоя:  $\frac{\partial \mathbf{a}_l}{\partial \mathbf{x}_l}$  - он позволит распространить ошибку на низлежащие слои.

Таким образом при обучении нам нужна только **очень локальная информация, содержащаяся в самом слое**. Т.е.:

- **нет необходимости знать как устроены соседние слои:** между слоями **очень простой интерфейс**
- т.е. **можно создать модульную архитектуру для слоев нейронной сети:** каждый модуль рассчитывает значение функции активации на основе выходов на прямом проходе и распространяет ошибку пришедшую на выходы на обратном проходе; все модули стандартным образом стыкуются друг с другом
- при модульной архитектуре граф нейронной сети может быть очень сложным, но его **расчет выполняется по одной простой и универсальной схеме**
- **внутри модули могут быть сложно устроены, это никак не меняет логику остальных модулей и всего процесса обучения** главное чтобы модуль корректно

## Дифференцируемое программирование и реализация обратного распространения ошибки

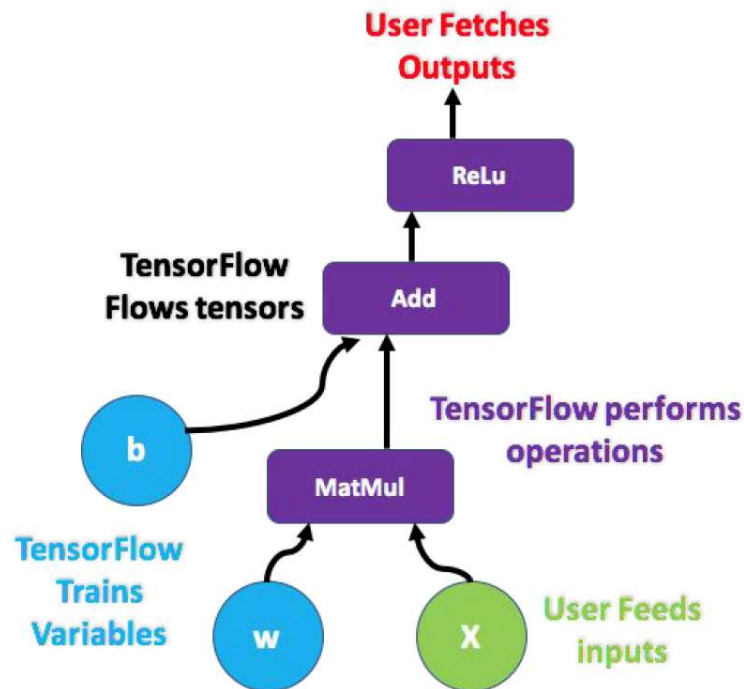
- [к оглавлению](#)

### Почему Tensor Flow?

Как реализовать **алгоритм обратного распространения ошибки** удобно для использования в задачах моделирования ИНС?

## Основная абстракция TensorFlow, PyTorch и других аналогичных библиотеках - **граф потока вычислений**.

- Рассматриваемые библиотеки обычно:
  - задают **граф потока вычислений** (формирует объект отложенных вычислений)
  - запускают **процедуру выполнения отложенных вычислений** и получает **результаты** вычислений (в т.ч. ошибку модели).
- Возможность в явном виде работать с графом потока вычислений дает большое преимущество для **автоматического решения задачи обратного распространения ошибки**, являющейся составляющей адачей обучения модели ИНС.

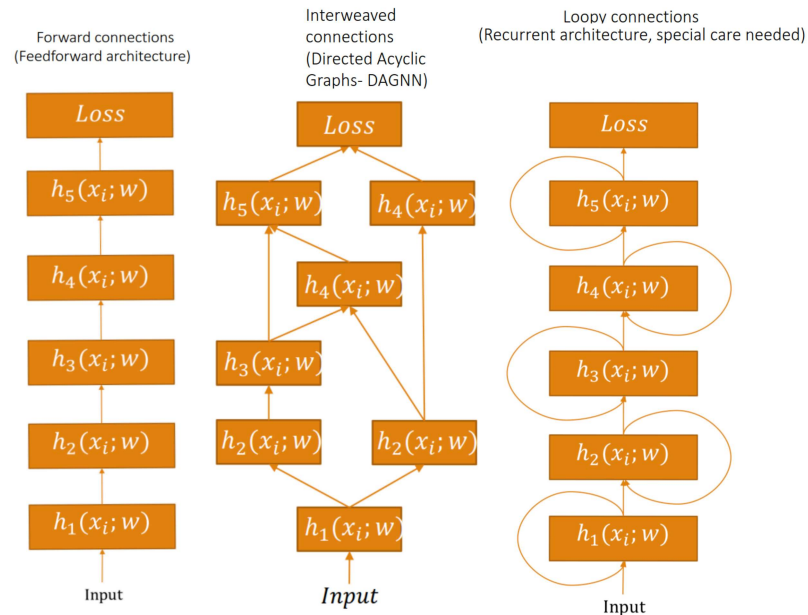


### Принцип устройства графа потока вычислений в TensorFlow

- Нейронная сеть это иерархия (она может быть простой и очень сложной) связанных (последовательно применяемых) функций слоев ИНС. Модель сети  $f_L$  может быть представлена как суперпозиция из  $L$  слоев  $h^i, i \in \{1, \dots, L\}$ , каждый из которых параметризуется своими весами  $w_i$ :

$$f_L(\mathbf{x}, \boldsymbol{\theta}) = f_L(\mathbf{x}, \mathbf{w}_1, \dots, \mathbf{w}_L) = h^L(h^{L-1}(\dots h^1(\mathbf{x}, \mathbf{w}_1), \dots, \mathbf{w}_{L-1}), \mathbf{w}_L)$$

- Вычисление функций слоев и взаимосвязи между слоями формируют граф потока вычислений в библиотеке моделирования ИНС.



### Примеры иерархий в нейронных сетях

- По сути, ИНС это композиция модулей, представляющих собой слои нейронной сети:
  - если сеть прямого распространения (feedforward), то все просто
  - если сеть является направленным ациклическим графом, то существует правильный порядок применения функций
  - в случае, если есть циклы, образующие рекуррентные связи, то существуют специальные подходы (будут рассмотрены позднее)

### Дифференцируемое программирование

**Дифференцируемое программирование** (differentiable programming) - парадигма программирования при которой программа (функция расчета значения) может быть продифференцирована в любой точке, обычно с помощью **автоматического дифференцирования**.

Это свойство позволяет использовать к программе **методы оптимизации основанные на расчете градиента**, обычно - **методы градиентного спуска**.

Дифференцируемое программирование используется в:

- глубоком обучении
- глубоком обучении комбинированном с физическими моделями в робототехнике
- специализированных методах трассировки лучей
- обработке изображений

Большинство фреймворков для дифференцируемого программирования использует граф потока вычислений определяющий выполнение программы и ее структуры данных.

Основные классы фреймворков для дифференцируемого программирования:

- статические** - они компилируют граф потока вычислений. Типичные представители: TensorFlow, Theano и др. Плюсы и минусы

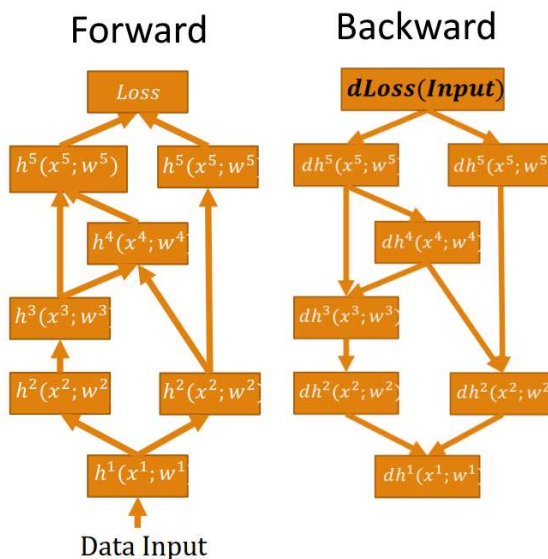


- могут использовать оптимизацию при компиляции
- легче масштабируются на большие системы
- статичность ограничивает интерактивность
- многие программы не могут реализовываться легко (в частности: циклы, рекурсия)
- **динамические** - динамически исполняют граф потока вычислений. Используют перегрузку операторов для записи. Типичные представители: PyTorch, AutoGradrFlow. Плюсы и минусы:
  - более простая и понятная запись программы
  - накладные расходы интерпретатора
  - невозможно использовать оптимизацию компилятора
  - хуже масштабируемость
- статическая на основе разбора промежуточного представления синтаксического разбора исходной программы. Пример фреймворк Zygote (язык программирования Julia).

### Прямой проход:

- Модули из графа обходятся один за одним начиная с узла входных данных и далее по мере готовности всех необходимых входных данных для очередного модуля, который еще не был обойден
- Расчет функций активации для каждого модуля по входным данным:  $a_l = h_l(x_l, w_l)$
- Промежуточные значения кэшируются, чтобы не рассчитывать их повторно (в сложном графе сети и при обратном проходе)
- Выходы одних модулей становятся входами других модулей:  $x_{l+1} = a_l$
- Последним модулем рассчитывается сумма потерь для входных данных

### Прямой и обратный проход процедуры обучения многослойной ИНС:



### Обратный проход:

- Сначала должен быть произведен прямой проход. На входе обратного прохода известна сумма потерь.
- Строится обратный порядок обхода графа зависимостей модулей.

- Модули из графа обходятся один за одним начиная с узла расчета функции потерь и далее по мере готовности всех необходимых входных данных для очередного модуля, который еще не был обойден
- Для каждого модуля рассчитываются якобиан функции активации по параметрам слоя  $\frac{\partial \mathbf{a}_l}{\partial \mathbf{w}_l}$  и якобиан функции активации по входным значениям слоя:  $\frac{\partial \mathbf{a}_l}{\partial \mathbf{x}_l}$
- По пришедшему в модуль градиенту ошибки (полученному из модулей использовавших результаты данного модуля на прямом проходе)  $\frac{\partial E}{\partial \mathbf{a}_l}$

рассчитывается:

- Градиент для шага градиентного спуска по параметрам модуля  $w_l$ :

$$\frac{\partial E}{\partial \mathbf{w}_l} = \left( \frac{\partial \mathbf{a}_l}{\partial \mathbf{w}_l} \right)^T \cdot \frac{\partial E}{\partial \mathbf{a}_l}$$

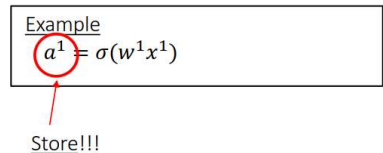
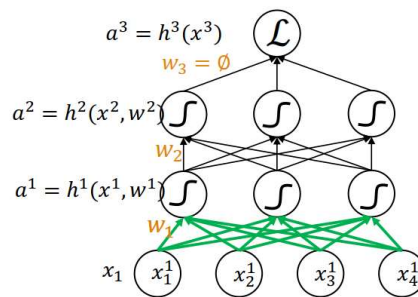
- Градиент ошибки, который передается в модули, поставившие данные в этот

модуль во время прямого прохода:  $\frac{\partial E}{\partial \mathbf{a}_{l-1}} = \left( \frac{\partial \mathbf{a}_l}{\partial \mathbf{x}_l} \right)^T \cdot \frac{\partial E}{\partial \mathbf{a}_l}$

### Пример: прямой проход, шаг 1

Forward propagations

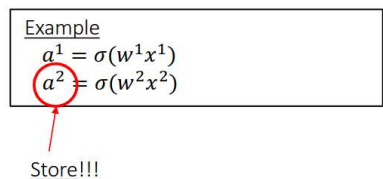
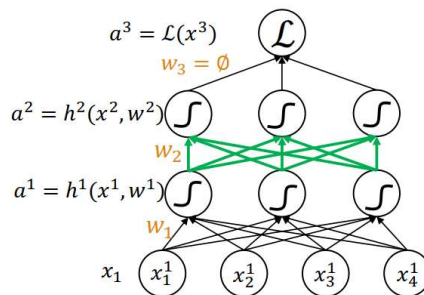
Compute and store  $a_1 = h_1(x_1)$



### Пример: прямой проход, шаг 2

Forward propagations

Compute and store  $a_2 = h_2(x_2)$



### Пример: прямой проход, шаг 3

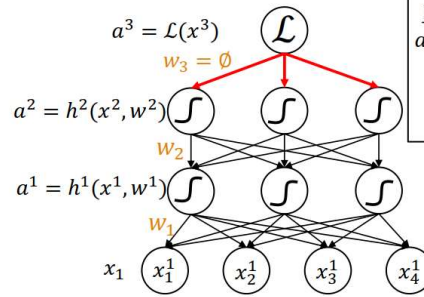
Forward propagations

Example

## Пример: обратный проход, шаг 1

Backpropagation

$$\frac{\partial \mathcal{L}}{\partial a^3} = \dots \leftarrow \text{Direct computation}$$

 ~~$\frac{\partial \mathcal{L}}{\partial w^3}$~~ 


Example

$$a^3 = \mathcal{L}(y, x^3) = 0.5 \|y - x^3\|^2$$

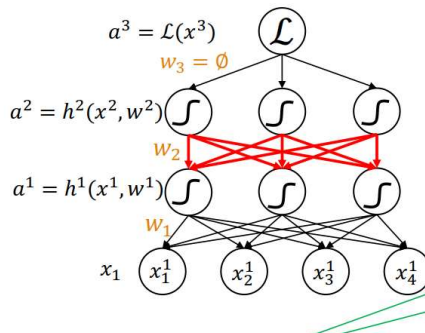
$$\frac{\partial \mathcal{L}}{\partial x^3} = -(y - x^3)$$

## Пример: обратный проход, шаг 2

Backpropagation

$$\frac{\partial \mathcal{L}}{\partial a^2} = \frac{\partial \mathcal{L}}{\partial a^3} \cdot \frac{\partial a^3}{\partial a^2}$$

$$\frac{\partial \mathcal{L}}{\partial w^2} = \frac{\partial \mathcal{L}}{\partial a^2} \cdot \frac{\partial a^2}{\partial w^2}$$



Stored during forward computations

Store!!!

Example

$$\mathcal{L}(y, x_3) = 0.5 \|y - x_3\|^2$$

$$a^2 = \sigma(w^2 x^2)$$

$$\frac{\partial \mathcal{L}}{\partial a^2} = \frac{\partial \mathcal{L}}{\partial x^3} = -(y - x_3)$$

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)(1 - \sigma(x))$$

$$\frac{\partial a^2}{\partial w^2} = x^2 \sigma(w^2 x^2) (1 - \sigma(w^2 x^2))$$

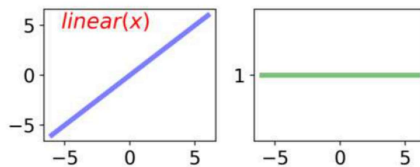
$$= x^2 a^2 (1 - a^2)$$

$$\frac{\partial \mathcal{L}}{\partial a^2} = -(y - x^3)$$

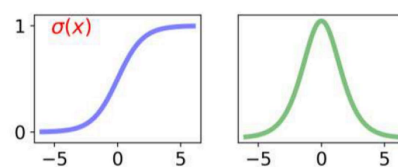
$$\frac{\partial \mathcal{L}}{\partial w^2} = \frac{\partial \mathcal{L}}{\partial a^2} x^2 a^2 (1 - a^2)$$

## Производные популярных функций активации

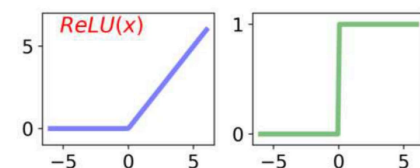
$$\text{Linear}(x) = x \xrightarrow{\frac{\partial \cdot}{\partial x}} 1$$



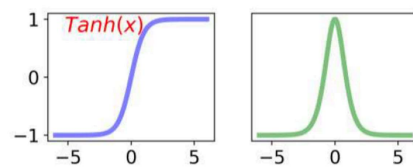
$$\sigma(x) = \frac{e^x}{1 + e^x} \xrightarrow{\frac{\partial \cdot}{\partial x}} \sigma(x) - \sigma^2(x)$$



$$\text{ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases} \xrightarrow{\frac{\partial \cdot}{\partial x}} \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \xrightarrow{\frac{\partial \cdot}{\partial x}} 1 - \tanh^2(x)$$

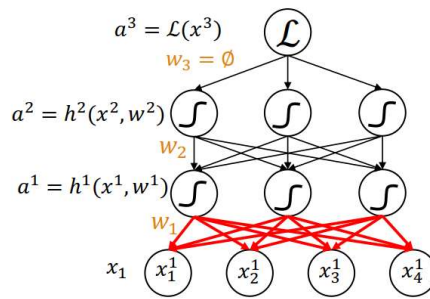


## Пример: обратный проход, шаг 3

Backpropagation

$$\frac{\partial \mathcal{L}}{\partial a_1} = \frac{\partial \mathcal{L}}{\partial a_2} \cdot \frac{\partial a_2}{\partial a_1}$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_1}$$



Computed from the exact previous

Example

$$\mathcal{L}(y, a^3) = 0.5 \|y - a^3\|^2$$

$$a^2 = \sigma(w^2 x^2)$$

$$a^1 = \sigma(w^1 x^1)$$

$$\frac{\partial a^2}{\partial a^1} = \frac{\partial a^2}{\partial a^2} = w^2 a^2 (1 - a^2)$$

$$\frac{\partial a^1}{\partial w^1} = x^1 a^1 (1 - a^1)$$

$$\frac{\partial \mathcal{L}}{\partial a^2} = \frac{\partial \mathcal{L}}{\partial a^2} w^2 a^2 (1 - a^2)$$

$$\frac{\partial \mathcal{L}}{\partial w^1} = \frac{\partial \mathcal{L}}{\partial w^1} x^1 a^1 (1 - a^1)$$

## Автоматическое дифференцирование в PyTorch

- [к оглавлению](#)

```
In [79]: # Пакет autograd обеспечивает автоматическое дифференцирование
# для всех операций с тензорами

# require_grad = True -> отслеживает все операции с тензором.
x = torch.randn(3, requires_grad=True)
y = x + 2

# y был создан в результате операции, поэтому у него есть атрибут grad_fn.
# grad_fn: ссылается на функцию, создавшую тензор
print(x) # создано пользователем -> grad_fn имеет значение None
print(y)
print(y.grad_fn)

tensor([-0.08521,  0.25389, -0.85220], requires_grad=True)
tensor([1.91479, 2.25389, 1.14780], grad_fn=<AddBackward0>)
<AddBackward0 object at 0x00000221B5628908>
```

```
In [80]: #Выполните больше операций над y
z = y * y * 3
print(z)
z = z.mean()
print(z)

tensor([10.99930, 15.24001,  3.95233], grad_fn=<MulBackward0>)
tensor(10.06388, grad_fn=<MeanBackward0>)
```

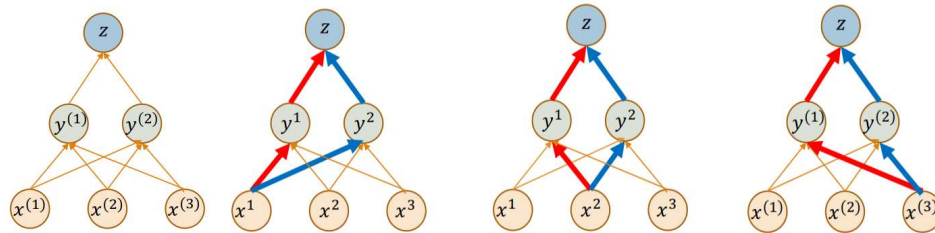
```
In [81]: # Давайте посчитаем градиенты с помощью обратного распространения ошибки
# Когда мы закончим наши вычисления, мы можем вызвать .backward() и автоматически
# Градиент этого тензора будет накоплен в атрибуте .grad.
# Это частная производная функции по w.r.t. тензор
```

```
z.backward()
print(x.grad) # dz/dx
```

```
# Вообще говоря, torch.autograd – это движок для вычисления векторно-якобиева
# Он вычисляет частную производную при применении правила цепочки
```

```
tensor([3.82959, 4.50777, 2.29560])
```

Примеры расчета градиента суперпозиции двух функций нескольких переменных:



$$\frac{dz}{dx^1} = \frac{dz}{dy^1} \frac{dy^1}{dx^1} + \frac{dz}{dy^2} \frac{dy^2}{dx^1} \quad \frac{dz}{dx^2} = \frac{dz}{dy^1} \frac{dy^1}{dx^2} + \frac{dz}{dy^2} \frac{dy^2}{dx^2} \quad \frac{dz}{dx^3} = \frac{dz}{dy^1} \frac{dy^1}{dx^3} + \frac{dz}{dy^2} \frac{dy^2}{dx^3}$$

Т.е. нам нужны градиенты по всем возможным путям (рассмотренным в обратном порядке) зависимостей переменных.

Запись этой же задачи в векторной нотации:

$$\bullet \quad \frac{dz}{dx} = \nabla_x(z) = \begin{pmatrix} \frac{\partial z}{\partial x_1} \\ \dots \\ \frac{\partial z}{\partial x_n} \end{pmatrix} = \left( \frac{dy}{dx} \right)^T \cdot \nabla_y(z) = J(\mathbf{y}(\mathbf{x}))^T \cdot \nabla_y(z) = J(\mathbf{y}(\mathbf{x}))^T \cdot \begin{pmatrix} \frac{\partial z}{\partial y_1} \\ \dots \\ \frac{\partial z}{\partial y_m} \end{pmatrix}$$

• Где  $J$  это Якобиан:

$$J(\mathbf{y}(\mathbf{x})) = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \dots & \frac{\partial y_m}{\partial x_n} \end{pmatrix}$$

```
In [82]: # Модель с нескалярным выходом:
# Если тензор нескалярный (более 1 элемента), нам нужно указать аргументы для
# указать аргумент градиента, который является тензором соответствующей формы.
# необходим для векторно-якобиева произведения

x = torch.randn(3, requires_grad=True)

y = x * 2
for _ in range(10):
    y = y * 2

print(y)
print(y.shape)

tensor([ 2596.08423, -432.97888, -2130.46240], grad_fn=<MulBackward0>)
torch.Size([3])
```

```
In [83]: v = torch.tensor([0.1, 1.0, 0.0001], dtype=torch.float32)
y.backward(v)
print(x.grad)

tensor([2.04800e+02, 2.04800e+03, 2.04800e-01])
```

Не позволяйте тензору отслеживать историю: например, во время нашего цикла обучения, когда мы хотим обновить наши веса, эта операция обновления не должна быть частью вычисления градиента.

- `x.requires_grad_(False)`
- `x.detach()`
- wrap in with `torch.no_grad()`:

```
In [85]: # .requires_grad_(...) изменяет существующий флаг на месте.

a = torch.randn(2, 2)
print(f'a.requires_grad = {a.requires_grad}')

b = ((a * 3) / (a - 1))
print(f'b.grad_fn = {b.grad_fn}')

a.requires_grad_(True)
print(f'a.requires_grad = {a.requires_grad}')

b = (a * a).sum()
print(f'b.grad_fn = {b.grad_fn}')

a.requires_grad = False
b.grad_fn = None
a.requires_grad = True
b.grad_fn = <SumBackward0 object at 0x00000221B56D26C8>
```

```
In [87]: # .detach(): получить новый Tensor с тем же содержимым, но без вычисления града  
a = torch.randn(2, 2, requires_grad=True)  
print(a.requires_grad)  
  
b = a.detach()  
print(b.requires_grad)
```

True

False

```
In [88]: # перенос 'с помощью torch.no_grad():'  
a = torch.randn(2, 2, requires_grad=True)  
print(a.requires_grad)  
  
with torch.no_grad():  
    print((x ** 2).requires_grad)
```

True

False

---



```

In [89]: # Back() накапливает градиент для этого тензора в атрибуте .grad.
# !!! Нужно быть осторожными при оптимизации!!!
# Используйте .zero_(), чтобы очистить градиенты перед новым шагом оптимизации

weights = torch.ones(4, requires_grad=True)

for epoch in range(3):
    # просто пример-пустышка
    # 'проход вперед'
    model_output = (weights*3).sum()

    model_output.backward()

    print(weights.grad)

    # оптимизируем модель, т.е. корректируем веса...
    with torch.no_grad():
        weights -= 0.1 * weights.grad

    #это важно! Это влияет на окончательный вес и производительность.
    weights.grad.zero_()

print(weights)
print(model_output)

# Оптимизатор имеет метод zero_grad()
# optimizer = torch.optim.SGD([weights], lr=0.1)
# Во время обучения:
# optimizer.step()
# optimizer.zero_grad()

tensor([3., 3., 3., 3.])
tensor([3., 3., 3., 3.])
tensor([3., 3., 3., 3.])
tensor([0.10000, 0.10000, 0.10000, 0.10000], requires_grad=True)
tensor(4.80000, grad_fn=<SumBackward0>)

```

---

Автоматическое выполнение обратного прохода с помощью `1.backward()` :



```

In [90]: # Модель линейной регрессии (с несколькими параметрами)
#  $f = X * w$ 

# Данные для обучения:
# признаки X: рассматривается 4 наблюдения (ось 0) и 2 признака (ось 1):

X = torch.tensor([[1., 40.],
                  [2., 30.],
                  [3., 20.],
                  [4., 10.]], dtype=torch.float32) # Size([4, 2])

# истинное значение весов (используется только для получения обучающих правил)
w_ans = torch.tensor([2., 1.], dtype=torch.float32)
# Y - правильные ответы:
Y = X @ w_ans

torch.set_printoptions(precision=5) # точность вывода на печать значений тензоров
print(f'w true value = {w_ans}, Y = {Y}')

# model (модель, в нашем случае: линейная регрессия)

# изначальное значение весов w
#!!! requires_grad=True
w = torch.tensor([0.0, 0.0], dtype=torch.float32, requires_grad=True)

# прямое распространение:
def forward(X):
    return X @ w # Size([4])

# Loss = MSE (функция потерь, в нашем случае: средняя квадратичная ошибка)
def loss(y, y_pred):
    return ((y_pred - y)**2).mean() # Size([1])

# Training
learning_rate = 0.0013
n_iters = 1000 + 1

# основной цикл:
for epoch in range(n_iters):
    # predict = forward pass
    y_pred = forward(X)

    # loss
    l = loss(Y, y_pred)

    #!!! backward pass
    # calculate gradients = backward pass
    l.backward()

    # update weights
    #w.data = w.data - learning_rate * w.grad
    with torch.no_grad():
        w -= learning_rate * w.grad

    # zero the gradients after updating

```

```
w.grad.zero_()
```

```
if epoch % 5 == 0:
    print(f'epoch {epoch}: w = {w}, y_pred = {y_pred}, loss = {l:.8f}\ngra
```

```
w true value = tensor([2., 1.]), Y = tensor([42., 34., 26., 18.])
epoch 0: w = tensor([0.16900, 2.21000], requires_grad=True), y_pred = tensor([0., 0., 0., 0.], grad_fn=<MvBackward>), loss = 980.00000000
gradient = tensor([-0.00032, -0.00026])
epoch 5: w = tensor([0.13813, 0.24422], requires_grad=True), y_pred = tensor([81.70274, 61.57523, 41.44772, 21.32021], grad_fn=<MvBackward>), loss = 646.58898926
gradient = tensor([-0.00032, -0.00026])
epoch 10: w = tensor([0.33966, 1.82453], requires_grad=True), y_pred = tensor([15.22920, 11.70164, 8.17407, 4.64651], grad_fn=<MvBackward>), loss = 427.49307251
gradient = tensor([-0.00032, -0.00026])
epoch 15: w = tensor([0.34364, 0.53338], requires_grad=True), y_pred = tensor([68.78386, 52.09385, 35.40385, 18.71384], grad_fn=<MvBackward>), loss = 283.42614746
gradient = tensor([-0.00032, -0.00026])
epoch 20: w = tensor([0.49880, 1.56850], requires_grad=True), y_pred = tensor([25.13912, 19.37721, 13.61531, 7.85340], grad_fn=<MvBackward>), loss = 188.61228943
gradient = tensor([-0.00032, -0.00026])
```

Использование встроенного оптимизатора `optimizer = torch.optim.SGD([w], lr=learning_rate)` и функции потерь `loss = nn.MSELoss()` :



```

In [91]: import torch
import torch.nn as nn

# Модель линейной регрессии (с несколькими параметрами)
#  $f = X * w$ 

#-----
# 0) Training samples

# Данные для обучения:
# признаки X: рассматривается 4 наблюдения (ось 0) и 2 признака (ось 1):
X = torch.tensor([[1., 40.],
                  [2., 30.],
                  [3., 20.],
                  [4., 10.]], dtype=torch.float32) # Size([4, 2])

# истинное значение весов (используется только для получения обучающих правил)
w_ans = torch.tensor([2., 1.], dtype=torch.float32)
# Y - правильные ответы:
Y = X @ w_ans

torch.set_printoptions(precision=5) # точность вывода на печать значений тензоров
print(f'w true value = {w_ans}, Y = {Y}')

#-----
# 1) Design Model: Weights to optimize and forward function

# изначальное значение весов w
w = torch.tensor([0.0, 0.0], dtype=torch.float32, requires_grad=True)

# model (модель, в нашем случае: линейная регрессия)
# прямое распространение:
def forward(X):
    return X @ w # Size([4])

#-----
# 2) Define Loss and optimizer

# callable function
loss = nn.MSELoss()

# Loss = MSE (функция потерь, в нашем случае: средняя квадратичная ошибка)
# def loss(y, y_pred):
#     return ((y_pred - y)**2).mean() # Size([])

learning_rate = 0.0013
optimizer = torch.optim.SGD([w], lr=learning_rate)

#-----
# 3) Training Loop
# основной цикл:
n_iters = 1000 + 1

for epoch in range(n_iters):
    # predict = forward pass

```

```

y_pred = forward(X)

# loss
l = loss(Y, y_pred)

# calculate gradients = backward pass
l.backward()

# update weights
optimizer.step()

# zero the gradients after updating
optimizer.zero_grad()

if epoch % 5 == 0:
    print(f'epoch {epoch}: w = {w}, y_pred = {y_pred}, loss = {l:.8f}\ngra

```

```

w true value = tensor([2., 1.]), Y = tensor([42., 34., 26., 18.])
epoch 0: w = tensor([0.16900, 2.21000], requires_grad=True), y_pred = tens
or([0., 0., 0., 0.], grad_fn=<MvBackward>), loss = 980.00000000
gradient = tensor([-0.00032, -0.00026])
epoch 5: w = tensor([0.13813, 0.24422], requires_grad=True), y_pred = tens
or([81.70274, 61.57523, 41.44772, 21.32021], grad_fn=<MvBackward>), loss =
646.58898926
gradient = tensor([-0.00032, -0.00026])
epoch 10: w = tensor([0.33966, 1.82453], requires_grad=True), y_pred = ten
sor([15.22920, 11.70164, 8.17407, 4.64651], grad_fn=<MvBackward>), loss
= 427.49307251
gradient = tensor([-0.00032, -0.00026])
epoch 15: w = tensor([0.34364, 0.53338], requires_grad=True), y_pred = ten
sor([68.78386, 52.09385, 35.40385, 18.71384], grad_fn=<MvBackward>), loss
= 283.42614746
gradient = tensor([-0.00032, -0.00026])
epoch 20: w = tensor([0.49880, 1.56850], requires_grad=True), y_pred = ten
sor([25.13912, 19.37721, 13.61531, 7.85340], grad_fn=<MvBackward>), loss
= 188.61228943
gradient = tensor([-0.00032, -0.00026])

```

Использование модели:

`torch.nn.Linear(in_features, out_features, bias=True)` Applies a linear transformation to the incoming data:  $y = xA^T + b$

Parameters:

- `in_features` – size of each input sample
- `out_features` – size of each output sample
- `bias` – If set to `False`, the layer will not learn an additive bias. Default: `True`



```
In [58]: # X_test = torch.tensor([[1], [2], [3], [4]], dtype=torch.float32) # torch.ten

X = torch.tensor([[1., 40.],
                  [2., 30.],
                  [3., 20.],
                  [4., 10.]], dtype=torch.float32) # Size([4, 2])

n_samples, n_features = X.shape

print(n_samples, n_features)

# n_samples, n_features = X_test.shape
# input_size = n_features
# output_size = n_features

class LinearRegression(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(LinearRegression, self).__init__()
        # define diferent layers
        self.lin = nn.Linear(input_dim, output_dim, bias=False)
    def forward(self, x):
        return self.lin(x)

model = LinearRegression(n_features, 1)

print(f'Prediction before training: f(5) = {model(X)}')
```

4 2

Prediction before training: f(5) = tensor([[ 6.43014],  
[ 4.13109],  
[ 1.83204],  
[-0.46701]], grad\_fn=<MmBackward>)

```
In [59]: # X_test.shape
```



```

In [55]: import torch
import torch.nn as nn

# Модель линейной регрессии (с несколькими параметрами)
#  $f = X * w$ 

#-----
# 0) Training samples

# Данные для обучения:
# признаки X: рассматривается 4 наблюдения (ось 0) и 2 признака (ось 1):
X = torch.tensor([[1., 40.],
                  [2., 30.],
                  [3., 20.],
                  [4., 10.]], dtype=torch.float32) # Size([4, 2])

print(f'X.shape = {X.shape}')
X_samples, X_features = X.shape

# истинное значение весов (используется только для получения обучающих правил)
w_ans = torch.tensor([2., 1.], dtype=torch.float32)
# Y - правильные ответы:
Y = X @ w_ans
print(f'Y.shape = {Y.shape}')
Y_features = 1

torch.set_printoptions(precision=5) # точность вывода на печать значений тензоров
print(f'w true value = {w_ans}, Y = {Y}')

#-----
# 1) Design Model, the model has to implement the forward pass!
# Here we can use a built-in model from PyTorch
input_size = n_features
output_size = n_features

# we can call this model with samples X
# model = nn.Linear(input_size, output_size)

class LinearRegression(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(LinearRegression, self).__init__()
        # define different layers
        self.lin = nn.Linear(input_dim, output_dim, bias=False)
    def forward(self, x):
        return self.lin(x)

model = LinearRegression(X_features, Y_features)

print(f'Prediction before training: f({X}) = {model(X)}')

# # model (модель, в нашем случае: линейная регрессия)
# # прямое распространение:
# def forward(X):
#     return X @ w # Size([4])

```

```

#-----
# 2) Define Loss and optimizer

# callable function
criterion = nn.MSELoss()

learning_rate = 0.0013
optimizer = torch.optim.SGD([w], lr=learning_rate)

#-----
# 3) Training Loop
# основной цикл:
n_iters = 1000 + 1

for epoch in range(n_iters):
    # Forward pass and Loss
    y_predicted = model(X)
    loss = criterion(y_predicted, Y)

    # Backward pass and update
    loss.backward()
    optimizer.step()

    # zero grad before new step
    optimizer.zero_grad()

    if epoch % 5 == 0:
        print(f'epoch {epoch}: w = {w}, y_pred = {y_pred}, loss = {l:.8f}\ngra

```

```

X.shape = torch.Size([4, 2])
Y.shape = torch.Size([4])
w true value = tensor([2., 1.]), Y = tensor([42., 34., 26., 18.])
Prediction before training: f(tensor([[ 1., 40.],
      [ 2., 30.],
      [ 3., 20.],
      [ 4., 10.]))) = tensor([[ -0.36370],
      [-0.80204],
      [-1.24038],
      [-1.67872]], grad_fn=<MmBackward>)
epoch 0: w = tensor([1.99996, 1.00000], requires_grad=True), y_pred = tensor([42.00007, 34.00001, 25.99994, 17.99988], grad_fn=<MvBackward>), loss = 0.00000001
gradient = tensor([ -130., -1700.])
epoch 5: w = tensor([1.99996, 1.00000], requires_grad=True), y_pred = tensor([42.00007, 34.00001, 25.99994, 17.99988], grad_fn=<MvBackward>), loss = 0.00000001
gradient = tensor([ -130., -1700.])
epoch 10: w = tensor([1.99996, 1.00000], requires_grad=True), y_pred = tensor([42.00007, 34.00001, 25.99994, 17.99988], grad_fn=<MvBackward>), loss =

```

## Спасибо за внимание!

## Технический раздел:

In [ ]:

In [ ]:

- И Введение в искусственные нейронные сети
  - Базовые понятия и история
- И Машинное обучение и концепция глубокого обучения
- И Почему глубокое обучение начало приносить плоды и активно использоваться только после 2010 г?
  - Производительность оборудования
  - Доступность наборов данных и тестов
  - Алгоритмические достижения в области глубокого обучения
    - Улучшенные подходы к регуляризации
    - Улучшенные схемы инициализации весов
    - (повтор) Усовершенствованные методы градиентного спуска
- Обратное распространение ошибки
  - Оптимизация
    - Стохастический градиентный спуск
    - Усовершенствованные методы градиентного спуска
- Введение в PyTorch

next qs line

next an line

next an line

next df line

next ex line

next pl line

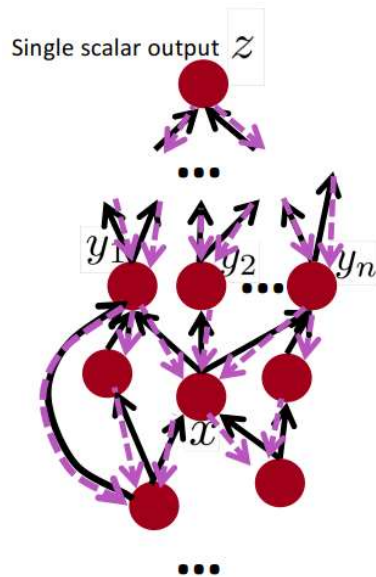
next mn line

next plmn line

next hn line

- Работа с графом потока вычислений нужна для того, чтобы решить **задачу обучения многослойной ИНС**. А эта задача требует после получения результатов и оценки ошибки **выполнения обратного прохода** дающего градиент ошибки для весов (параметров) модели и последующей процедуры оптимизации весов.

## Back-Prop in General Computation Graph



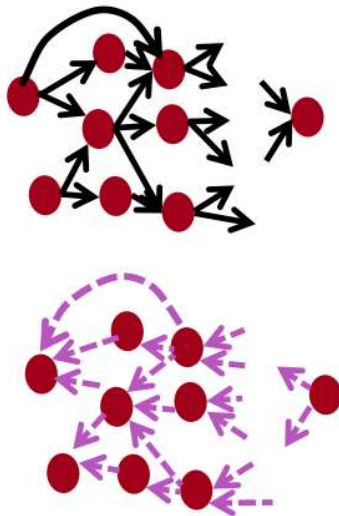
1. Fprop: visit nodes in topological sort order
    - Compute value of node given predecessors
  2. Bprop:
    - initialize output gradient = 1
    - visit nodes in reverse order:
      - Compute gradient wrt each node using gradient wrt successors
- $\{y_1, y_2, \dots, y_n\} = \text{successors of } x$

$$\frac{\partial z}{\partial x} = \sum_{i=1}^n \frac{\partial z}{\partial y_i} \frac{\partial y_i}{\partial x}$$

Done correctly, big  $O()$  complexity of fprop and bprop is **the same**

In general our nets have regular layer-structure and so we can use matrices and Jacobians...

## Automatic Differentiation

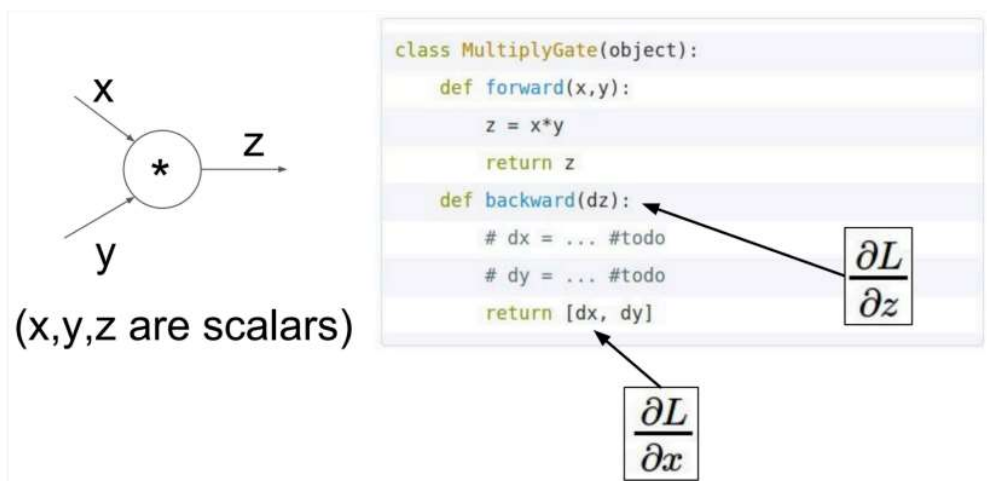


- The gradient computation can be automatically inferred from the symbolic expression of the fprop
- Each node type needs to know how to compute its output and how to compute the gradient wrt its inputs given the gradient wrt its output
- Modern DL frameworks (Tensorflow, PyTorch, etc.) do backpropagation for you but mainly leave layer/node writer to hand-calculate the local derivative

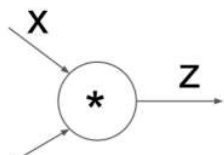
## Backprop Implementations

```
class ComputationalGraph(object):
    #...
    def forward(inputs):
        # 1. [pass inputs to input gates...]
        # 2. forward the computational graph:
        for gate in self.graph.nodes_topologically_sorted():
            gate.forward()
        return loss # the final gate in the graph outputs the loss
    def backward():
        for gate in reversed(self.graph.nodes_topologically_sorted()):
            gate.backward() # little piece of backprop (chain rule applied)
        return inputs_gradients
```

### Implementation: forward/backward API



## Implementation: forward/backward API



```
class MultiplyGate(object):  
    def forward(x,y):  
        z = x*y  
        self.x = x # must keep these around!  
        self.y = y
```

In [ ]: |